

Systems Engineering

Crash Course — Study Guide

Lectures 3 – 6 · ENGR 302

Evidence, needs, context diagrams, requirements, verification, traceability

Stage 1	Problem definition	Stakeholders · Evidence · User Needs · Stakeholder Reqs · Context Diagram
Stage 2	System level	System Concept · System Requirements · Physical I/O Diagram (subsystems)
Stage 3	Component level	Technical Requirements · Physical I/O with selected components
Verification	Going back up	Evidence · Method · Pass Criterion · Traceability matrix
Validation	At the top	Did we build the right thing? Test against user needs.

The BTC Case Study

Everything in lectures 3–6 is built around one real problem: BTC already sold a box that tracked where a truck was. A logistics customer wanted to know what cargo was on the trailer — not just where the truck was. A university capstone team built a prototype over one year.

The system chain

One per pallet	Reads the tags	Collects, formats	GPS + GSM radio	Customer views here

KEY FACT

The last link (MCU to HCP65-G via GSM) never worked. Data had to be loaded by hand. This is why several verifications in the report described the design intent rather than what was actually built.

Stage 1 · Problem Definition

Stage 1 is about understanding the problem before touching any design. You are not allowed to name a technology. You are not designing anything yet.

Evidence, Needs, and Solutions (E / N / X)

Every statement a stakeholder makes is one of three things:

E — Evidence	N — Need	X — Neither (solution)
Something a person actually said or did. Attributed, verbatim, written down as spoken. Example: "If I have to stop and scan every pallet I am doing half the loads." — forklift operator	What someone must be able to do. No solution in it. No technology. Example: "The loader must be able to complete a load without stopping to record what went on."	Already names a technology or mechanism. Not a need — skip ahead to design. Example: "The system needs an RFID reader mounted at the trailer door."

SEQUENCE

Evidence → Needs → Requirements. Gather evidence first (what stakeholders actually said). Extract needs from it (what they must be able to do). Then write requirements to satisfy those needs. Running this backwards produces solutions before the problem is understood.

The Eight-Category Stakeholder Sweep

• Users — people who physically operate it • Beneficiaries — people whose work improves • Maintainers — people who repair, calibrate, update • Safety and compliance — legally responsible if something goes wrong

• Owners and sponsors — people paying for it • Integrators — systems/teams that must interface with yours • Affected non-users — people constrained by the system • Lifecycle stakeholders — people interacting in future phases

RULE

Name roles, not people. 'Forklift operator', not 'Bilal'. Roles survive staff changes. Requirements tied to a named individual become orphaned the day they leave.

The Context Diagram

The context diagram shows where the system ends and the world starts. You draw two versions:

As-Is diagram

To-Be diagram

The world as it exists today, before any system is built. No system of interest — it does not exist yet. Shows external entities and what crosses between them. BTC As-Is: 7 entities, 7 connections, nobody in the middle. Entities: Loader · Trailer · Pallets and cargo · Driver · Receiving plant · BTC vehicle tracking system · Logistics customer

Identical to the As-Is, with exactly ONE box added: the system you are building. Name that box by what it achieves, in three words or fewer, with no technology name. Yes: "Cargo location record" · "Cargo visibility service" No: "RFID tracking system" · "Trailer scanner" Every external entity is unchanged. Only one box was added. That is the entire contribution.

Classifying entities: I / E / A

- Internal (I) — part of what you are designing. The system of interest itself.
- External (E) — exists and interacts, but you cannot change it. The radio regulator constrains you whatever you build. Exists whether or not you build anything.
- Answer (A) — only exists if you choose a particular solution. RFID reader, GPS satellite, GPRS network, barcode scanner are all Answers in the BTC As-Is. The other three only exist once you have chosen a particular solution.

ORPHAN NEED

A need with no evidence source. Do not delete it. Go and ask someone. It is a question nobody has asked yet. In the BTC case, need C (the system must survive being washed down) had no extract — nobody had said it, but it was still real.

Stage 1–2 · Stakeholder Requirements

A requirement is the system's language: what the system must do, with a threshold you can test. Moving from a need to a requirement is not permission to start designing. The last row — 'no solution' — is the one people get wrong.

	Need	Requirement
Whose sentence	The person's	The system's
What it describes	What someone must be able to do	What the system must do about it
Language	Theirs	Ours
Measurement	None	A threshold, or an observable event
Solution	None	Still none

The Four Quality Criteria (D / V / C / OK)

Label	Name	The question to ask	Typical failure
D	Design independent	Could someone satisfy this more than one way?	Names a technology or mechanism
V	Verifiable	Is there a number, or something you could watch happen?	No threshold — 'up to', 'little to no', 'unusable'
C	Correct	Does any arithmetic in it hold?	Wrong numbers — e.g. 33 bytes × 1000 ≠ 3.02 KB
OK	Complete	What could go wrong that nothing here covers?	Gaps in the requirement set, not the sentence

Common Failures — two you will see over and over

• "up to 5 metres" — WRONG DIRECTION • A reader that manages one centimetre satisfies this. • The word you want is at least. • Requirement should say: 'at a distance of greater than or equal to 5.0 metres'

• "little to no operator input" — NO THRESHOLD • Two people would not agree on whether it passed. • Pick one of the two words and mean it. • Requirement should say: 'shall require no action from the driver or the loader during loading'

The Ten BTC Requirements — Quick Scorecard

#	Requirement (shortened)	Verdict	Why
1	Allows RFID module to communicate with GPS tracking system	D	Names two technologies and an architecture
2	Stores data from OEM RFID module for communication to GPS unit	D	Same problem, plus names a supplier's module
3	Stores RFID tag data with GPS location and time for transmission to web server	D	Three technologies in one sentence

4	Detects whether product is entering or exiting the truck	V	Solution-neutral, but no frequency threshold — nothing can fail it
5	Utilizes the vehicle's power to operate	D	A design choice presented as a requirement
6	Has a backup power source	D	Names the answer, not the need
7	Will queue movements when unusable (absent or low) GSM signal present	V	'Unusable', 'low', 'reliable' have no thresholds
8	Requires little to no vehicle-operator input	V	'Little to no' cannot be tested
9	Capable of storing at least 1000 tags of data	OK	Measurable, threshold, no technology. Best of the ten.
10	Can read RFID tags at a range of up to 5 metres	V	'Up to' — one centimetre satisfies this

Stage 2–3 · Verification

Verification answers: did we build it right? Every verification must have all three parts. A criterion written after the test is worthless — you already know the answer. A statement of intent is not a verification.

The Three Parts of a Verification

Something that exists afterwards that anyone can look at. Example: 'count of successful reads'	The named approach. One of five methods. Example: 'Test'	Decided before the test, not after. Example: 'Pass: 100 of 100'

Worked example: Requirement: 'The system shall identify an item at a distance of greater than or equal to 5.0 metres from the reading point.'

GOOD VERIFICATION Place a tagged item at 5.0 m and read it 100 times. Evidence: count of successful reads. Pass: 100 of 100. [Method: Test]

The Five Methods

Method	In one line	Use when	BTC example
Test	Run it, measure with instruments	You need a number from the real system	Read 100 tags at 5.0 m, count successes
Inspection	Look at it, without running it	A document or label answers the question	Check reader certificate says ISO 18000-6C
Analysis	Calculate it	Hand-calculation or reasoning is sufficient	33 bytes × 1000 tags vs fitted memory
Demonstration	Watch it work, no instruments	Behaviour is observable, no measurement needed	Load trailer, observe; pass: nothing done for system
Modelling	Simulate it	Real system too expensive, dangerous, or not yet built	Signal coverage across a route before hardware exists

Four Ways a Verification Says Nothing

Failure mode	Example from BTC report	What it lacks
Naming the part	"Requirement fulfilled by RFID reader from Alien Technologies"	No action taken. Nothing to look at afterwards.
Describing the design	"Data it receives is sent via GSM to BTC server"	Written from intent, not reality. The link never worked.
Testing the wrong property	V3 tested storage when the real problem was read accuracy	Real evidence, but the requirement asked for something else.
No criterion	"We have verified that it works properly"	Nobody agreed beforehand what 'properly' meant.

V5 PARADOX V5 in the BTC report had all three parts (evidence: datasheet figures, method: analysis, criterion: 3.02 KB from the requirement). But the requirement was wrong. 33 bytes × 1000 tags = 33 KB, not 3.02 KB. A perfect verification of a defective requirement.

Traceability Matrix

A traceability matrix maps requirements (rows) to verifications (columns). You tick a cell only if the verification would actually catch a failure of that requirement. Mentioning the same words is not enough.

What the Matrix Reveals

- Empty row — requirement with no verification: an orphan requirement. Nobody will ever check it.
- Too many ticks in one column — verification is being borrowed. It probably addresses one requirement and is cited for the rest.

- Lumpy coverage — some requirements over-tested, some not at all. Nobody chose this; it is what happens when the matrix is written entry by entry at the end.
- The matrix takes ten minutes to build and finds things that reading the document line by line does not.

BTC Traceability Matrix — What went wrong

R e q	Requirement (shortened)	V 1	V 2	V 3	V 4	V 5	V 6	Issue
1	RFID communicates with GPS	✓					✓	V1 & V6 name parts; communication itself never tested
2	Stores data from OEM RFID module			✓				V3 only checks storage/format
3	Stores tag, location, time, direction		✓	✓			✓	Best covered row
4	Detects entering or exiting			✓				V3 checks direction 50–67% of time — no threshold
5	Uses vehicle power				✓			Only backup was checked; vehicle supply never measured
6	Has backup power source				✓			V4 confirms connection, not function
7	Queues when signal unusable							EMPTY — never verified
8	Little to no operator input							EMPTY — never verified
9	Stores at least 1000 tags					✓		Sound method — defective requirement (33KB ≠ 3.02KB)
10	Reads tags up to 5 metres	✓						Range never measured; read rate never specified

Quick Reference — Things to Know Cold

Key definitions

Evidence	A verbatim, attributed statement — something a person actually said or did.
User need	What someone must be able to do. No solution in it. No technology.
Orphan need	A need with no evidence source. Do not delete it — go and ask someone.
Requirement (D)	Design creeps in — names a technology or mechanism.
Requirement (V)	Cannot be tested — no threshold anyone can measure.
Requirement (C)	Factually wrong — the arithmetic does not hold.
Requirement (OK)	Nothing wrong with it. Design-independent, verifiable, correct, complete.
Verification	Evidence + Method + Pass Criterion (written before the test).
Orphan requirement	A requirement with no verification — an empty row in the matrix.
Borrowed verification	A verification cited against requirements it does not actually test.

Validation	Did we build the right thing? Test against user needs, not requirements.
-------------------	--

Verification method — in one line each

Test	Run it under controlled conditions and record numbers.
Inspection	Look at it without running it — configuration, labels, documents.
Analysis	Calculate it — hand calculations, margin analysis.
Demonstration	Operate the real system, no instruments, just watch it work.
Modelling	Simulate it — when the real system is dangerous, expensive, or not yet built.

Things the quiz tests your judgment on

- Classifying a statement as E, N, or X — the test is whether it names a solution.
- Classifying context diagram entities as I, E, or A — does it exist before any solution is chosen?
- Naming the system box — purpose, not mechanism. 'Cargo visibility service', not 'RFID tracker'.
- Diagnosing a requirement — apply D, V, C, OK one at a time. 'Up to' fails V. Technology names fail D.
- Judging a verification — look for all three parts. A missing criterion is common. 'Works properly' has none.
- Reading a traceability matrix — empty rows are unverified requirements. A column with too many ticks is probably borrowed.
- The V5 paradox — a verification can be technically complete and still verify a wrong requirement.
- Verification vs validation — verification is against requirements; validation is against user needs.